Engineering Backlog & Release Plan

VERSION 1.1 | 06 NOVEMBER 2026

400 COFFEE

Engineering Backlog & Release Plan

12-sprint delivery roadmap, epics, acceptance criteria and release gates

Document status: Professional working draft for stakeholder review. Open decisions and assumptions are explicitly identified.

Contents

1. Delivery approach

2. Sprint roadmap

3. Epic acceptance criteria

4. Definition of Ready / Done

5. Test strategy

6. Release gates and rollback

7. Key delivery risks

1. Delivery approach

Cadence 12 two-week sprints as an initial planning baseline. Re-estimate after Sprint 0; actual dates and team capacity are not yet committed.

Method Vertical slices with demonstrable increments, automated tests and stakeholder review. Maintain a prioritized product backlog and Definition of Ready/Done.

Team model Product owner/business representative, UX, backend, frontend, QA/automation, DevOps/security. Roles may be combined in a small team, but ownership must remain explicit.

2. Sprint roadmap

Sprint 0 — Discovery & foundations Validate workflows, menu/data, payment evidence, roles, tax/invoice assumptions, UX prototype, architecture decisions, threat model, acceptance criteria, environments and delivery plan.

Sprint 1 — SaaS foundation Tenant/outlet model, identity, roles, authorization, CI/CD, database migrations, audit baseline, API conventions and automated tenant isolation tests.

Sprint 2 — Catalog Categories, products, modifiers, prices, availability, tax/charge configuration, station routing and catalog administration.

Sprint 3 — POS Cashier order workspace, cart, modifiers, order type, server totals, order lifecycle, idempotency, receipt baseline and order search.

Sprint 4 — Split KDS Beverage/food station tickets, live queue, ticket timers, start/ready actions, parent readiness aggregation, reconnect and queue recovery.

Sprint 5 — Payments Payment attempts/ledger, cash, UPI declaration, manual verification, duplicate reference protection, partial/overpayment exceptions, refunds and audit.

Sprint 6 — Customer QR Public menu, secure QR context, guest cart, server quote, order submission, configurable release policy, tracking token and customer status.

Sprint 7 — Billing, shifts & exceptions Receipts, shift start/close, reconciliation, exception center, cancellation/refund authority and audit review.

Sprint 8 — Owner dashboard Metric definitions, sales/collections dashboard, product mix, delay metrics, date filters and export baseline.

Sprint 9 — SaaS onboarding & hardening Tenant onboarding, plan/entitlement, platform admin, security review, privacy controls, monitoring, backup/restore and support runbooks.

Sprint 10 — Inventory foundation Ingredients, recipes/BOM, units, stock movements, stock counts, adjustments and permission model.

Sprint 11 — Consumption & analytics Recipe consumption, wastage, variance, inventory reports and pilot analytics; validate data quality before advanced insights.

3. Epic acceptance criteria

EPIC A — Tenant/security All tenant-owned APIs scope by authenticated membership; negative tests cover cross-tenant reads/writes, exports, event subscriptions and public tokens.

EPIC B — POS/order Server computes totals; duplicate idempotency key returns original result; order state transitions reject invalid commands; audit captures actor and reason.

EPIC C — KDS Every accepted order routes each eligible item to exactly one configured station; station reconnect recovers queue; parent ready only after all required station work is ready.

EPIC D — Payments Declared payment remains unverified; authorized verification records amount/reference/verifier; duplicate references blocked; refunds are distinct ledger entries.

EPIC E — QR Guest can browse and submit online; policy snapshot preserved; price/availability revalidated; tracking token is unguessable and scoped.

EPIC F — Reporting Metric formulas documented; totals reconcile to ledger; filters use outlet timezone consistently; exports respect tenant scope.

4. Definition of Ready / Done

Ready User story has actor, problem, acceptance criteria, design/API dependencies, data/security impact, test approach and estimate.

Done Code reviewed; tests pass; API docs updated; telemetry and audit added where needed; accessibility reviewed; migration and rollback considered; no critical/high defects; demoed in staging; support notes updated.

5. Test strategy

Unit Pricing, tax/charge calculation, state machine, station routing, payment allocation, permissions and metric formulas.

Integration PostgreSQL persistence, transactional outbox, idempotency, payment ledger, public QR token, event delivery and backup restore.

E2E Cashier, QR, split kitchen, payment verification, partial payment, duplicate reference, cancellation, shift close, tenant isolation and reconnect scenarios.

Performance 3× measured peak order submissions; concurrent KDS clients; reconnect storm; report queries; p95/p99 latency and database contention.

Resilience Network drop during order submit; duplicate retry; event broker delay; client refresh; server restart; device replacement; failed notification.

6. Release gates and rollback

Pilot gate No lost acknowledged orders; no duplicate fulfillment; no false paid status; accurate balances; complete audit history; KDS event recovery; tenant isolation; peak-hour simulation; staff sign-off.

Production SaaS gate Self-service onboarding; tested restore; monitoring/alerts; incident runbooks; privacy/legal review; support process; subscription/entitlement correctness; penetration/security review.

Rollback Use feature flags and backward-compatible migrations. For data migrations, define forward-fix and restore strategy before release. Do not roll back by discarding accepted orders or ledger entries.

7. Key delivery risks

UPI integration Static QR requires manual verification; provider APIs may be unavailable. Mitigate with explicit manual process and no false automated status.

Operational adoption Staff may bypass or resist new steps. Mitigate through observation, minimal taps, training and floor support.

Internet/hardware Wi-Fi or device failures can interrupt service. Mitigate with local draft persistence, reconnect recovery and device qualification.

Scope growth Inventory, loyalty, AI and multi-outlet can distract from core reliability. Keep them outside pilot release gates.

Metric ambiguity Gross sales and cash collected may be conflated. Define metric formulas and reconcile against ledger.

VERSION 1.1 — CUSTOMER EXPERIENCE ADDENDUM

Version 1.1 addendum — 06 November 2026. This addendum supersedes any earlier customer-facing UX description that implied a conventional QR menu/dashboard as the primary customer experience. The immersive Virtual Café is the approved customer-side direction. Existing staff POS, KDS, owner and platform-admin interfaces remain separate workstreams and are not redesigned by this customer-side change.

4.1 Customer-side workstream

CX-S0 Discovery and assets: confirm final menu, brand assets, manager character design, photography, 3D pipeline, payment provider, WhatsApp BSP, pickup/dine-in rules and consent text. CX-S1 Foundation: PWA shell, routing, session, API client, design tokens, responsive baseline, accessibility and feature flags. CX-S2 Scene: optimized 3D café environment, manager rig/animations, camera choreography, loading, 2D fallback and reduced-motion. CX-S3 Diary menu: categories, catalog API, search/filter, availability, image loading, price display. CX-S4 Customizer/basket: modifiers, variants, quantity, instructions, server quote, persistence. CX-S5 Identity/review: name/mobile/consent, validation, review, edit/back, idempotent submit. CX-S6 Order handoff: backend order create, cashier acceptance policy, status screen and reconnect recovery. CX-S7 Payments: provider adapters, UPI/QR/card handoff, webhook verification, due-at-counter path, failure/expiry/retry. CX-S8 Bill payment: secure bill lookup and outstanding balance payment. CX-S9 Notifications: WhatsApp templates, consent, transactional outbox, delivery status/retry. CX-S10 QA/performance: device matrix, accessibility, security, load and visual regression. CX-S11 pilot rollout: analytics, staff training, support runbook and controlled release.

4.2 Engineering non-negotiables

Do not implement the customer journey as a static Figma-like screen or single long component. Build reusable components, typed state transitions, API contracts, error boundaries and tests. No secrets in client bundle. No trusting client totals. No fake paid state. No direct database access from browser. No hardcoded production menu or prices. No WhatsApp success toast without provider response. No fake order sync; mock adapter must be clearly labelled and replaceable. Keep staff/POS/KDS code outside this scope unless needed for contract tests.

4.3 Definition of done

Every story includes implementation, unit tests, component tests, API contract tests, accessibility check, responsive check, error/loading/empty states, telemetry with PII scrubbing, documentation and acceptance evidence. All critical payment/order tests pass. Visual QA is performed at mobile, tablet and desktop breakpoints. Performance budget is measured on representative mid-range Android hardware, not only a developer workstation.

DARK CINEMATIC CAFÉ-TECH | VERSION 1.0 | 06 NOVEMBER 2026